Ok, mi sembra che le sezioni volumes e mounts sono quelle piu articolate tra tutte quelle previste per i files runtime.yaml ovvero environment, env_files, ports etc. Non c'e' il rischio di dover implementare un interprete dello schema un po troppo complesso ?

Sì, il rischio c'è. La proposta precedente stava diventando troppo articolata: aggiungendo `lifecycle`, strategie di preparazione, hook, protezioni, riferimenti e regole di composizione, rischiamo di costruire un piccolo orchestratore anziché un adattatore tra configurazione e Docker.

La distinzione da mantenere è questa: `runtime.yaml` **descrive il runtime**, mentre gli script esistenti continuano a gestire la preparazione delle risorse. Non è necessario trasferire tutta la loro logica nello schema.

1. Dove nasce la complessità

`mounts` è relativamente semplice se descrive soltanto:

```
tipo + sorgente + destinazione + sola lettura
```

Diventa complesso quando deve anche stabilire:

- come creare il volume;
- come inizializzarlo;
- quale UID/GID applicare;
- quale hook chiamare;
- quando ripararlo;
- chi può sovrascriverlo;
- quando eliminarlo.

Queste sono operazioni di gestione delle risorse, non proprietà del mount.

Anche riprodurre una semantica generale di merge sarebbe oneroso: Compose stesso ha regole diverse per mapping, liste e risorse come porte e volumi. Non conviene implementare una seconda versione di queste regole nel nostro resolver.[1]

2. Semplificazione consigliata per la prima versione

Manteniamo i due livelli approvati:

```
langs/Python/runtime.yaml
<PROJECT_DIR>/.docker-dev/runtime.yaml
```

Ma riduciamo lo schema a dichiarazioni direttamente traducibili nei due backend.

File di linguaggio

```yaml
schema_version: 1

mounts:
  - type: bind
    source: "${context.project_dir}"
    target: /app

  - type: volume
    source: "${context.volume_name}"
    target: /app/.venv

  - type: volume
    source: uv-python
    target: "/home/${context.host_user}/.local/share/uv/python"
```

In questa versione:

- `source` di un volume è il nome Docker effettivo, dopo la risoluzione;
- non ci sono chiavi logiche da collegare a una seconda sezione;
- non ci sono `lifecycle`, `preparation`, `strategy` o hook nel file;
- `read_only` è facoltativo, con default `false`.

Il file dichiara dove montare le risorse. Non dichiara come prepararle.

File del progetto

```yaml
schema_version: 1

environment:
  APP_MODE: development

ports: []

mounts:
  - type: bind
    source: ./datasets
    target: /datasets
    read_only: true
```

Il progetto aggiunge configurazione senza dover conoscere il funzionamento del virtualenv Python.

3. Possiamo evitare la sezione `volumes` nel formato comune

Per la prima versione, sì: la eliminerei da `runtime.yaml`.

La lista dei mount contiene già i nomi dei volumi da usare. Il renderer Compose può ricavare automaticamente da essa la sezione top-level `volumes`.

Per esempio, dal mount risolto:

```
type: volume
source: venv.dev.python.foo
target: /app/.venv
```

il renderer genera:

```
services:
  dev:
    volumes:
      - type: volume
        source: runtime-volume-1
        target: /app/.venv

volumes:
  runtime-volume-1:
    external: true
    name: venv.dev.python.foo
```

La chiave `runtime-volume-1` è interna al documento generato: l'utente non deve dichiararla né gestirla. Compose distingue la chiave del volume nel modello dal nome effettivo della risorsa Docker.[2]

Questo elimina dal nostro schema:

- riferimenti incrociati tra `mounts` e `volumes`;
- nomi logici aggiuntivi;
- politiche di lifecycle configurabili;
- un intero livello di validazione.

4. Chi crea e inizializza i volumi?

Gli script di setup, come già avviene oggi.

Per Python:

```
10.setup-volumes.sh
    ├─ prepara il volume virtualenv
```

```
    └─ prepara uv-python
```

Il resolver fornisce i nomi risolti attraverso il contesto o il piano; lo script conserva la conoscenza specifica dell'inizializzazione.

Per la prima versione proporrei una regola unica:

> Tutti i volumi nominati montati nel container di sviluppo devono esistere dopo la fase di setup e prima dell'esecuzione.

Quindi:

- volumi standard: preparati dallo script del linguaggio;
- volumi aggiuntivi del progetto: inizialmente devono essere già disponibili;
- volume mancante: errore chiaro, non creazione implicita diversa tra legacy e Compose.

La creazione o inizializzazione automatica dei volumi aggiuntivi può essere introdotta in seguito come funzionalità separata, se emerge una necessità concreta.

Questa regola riguarda il container `dev`. I volumi di PostgreSQL o altri servizi restano nella configurazione e negli hook del servizio, senza obbligare `runtime.yaml` a descriverne la gestione.

5. Anche la composizione può essere molto semplice

Non serve un sistema generale di ereditarietà.

Sezione	Regola proposta
`environment`	Merge per chiave; il progetto prevale
`env_files`	Lista del linguaggio seguita da quella del progetto
`ports`	Aggiunta, con controllo dei conflitti
`mounts`	Aggiunta; destinazione duplicata → errore
`extra_hosts`	Merge per nome host; il progetto prevale
`shm_size`	Il progetto prevale se specificato
`resources`	Override per campo supportato

Per i mount standard non serve neppure `protected: true`: se nella prima versione ogni destinazione duplicata produce errore, il progetto non può sostituirli implicitamente.

Restano consentiti i mount annidati previsti, come:

```
/app
/app/.venv
```

Non implementiamo quindi “override arbitrari dei mount”, rimozione di elementi ereditati o operatori speciali di merge.

6. Quanto codice applicativo rimane?

Il resolver deve fare soltanto queste operazioni:

1. leggere i due YAML;
2. controllare chiavi e tipi ammessi;
3. risolvere un insieme limitato di riferimenti al contesto;
4. rendere assoluti i percorsi relativi;
5. comporre le sezioni secondo regole fisse;
6. rilevare conflitti;
7. produrre il modello normalizzato.

Poi:

```
modello normalizzato
    ├─ traduttore docker run
    └─ renderer Compose
```

Questa è una quantità di logica giustificata dalla necessità di supportare due backend. Il rischio cresce se aggiungiamo un linguaggio di espressioni, hook dinamici, lifecycle configurabili e merge sofisticati.

7. Alternativa ancora più semplice: solo Compose

Se in futuro decidessi di usare Compose anche per il solo container di sviluppo, potresti ridurre ulteriormente la traduzione fra backend. Compose permette infatti di eseguire un servizio come comando one-off, con rimozione all’uscita e pubblicazione delle porte mediante `--service-ports`.[3]

Ma sarebbe una scelta architetturale distinta: non è necessaria per questa prima versione, che vuole preservare il percorso legacy.

Raccomandazione per la CR

Ridimensionerei la proposta così:

- mantenere i due file `runtime.yaml`;
- mantenere un elenco semplice `mounts`;
- eliminare per ora la sezione comune `volumes`;
- lasciare creazione e inizializzazione agli script di setup;
- usare regole di composizione fisse e limitate;
- trattare tutti i volumi di `dev` come esterni nel Compose generato;

- rinviare lifecycle, hook dichiarativi e override avanzati finché non servono davvero.

In questo modo conserviamo la fonte comune per legacy e Compose, senza trasformare il formato runtime in un sistema di orchestrazione autonomo.

⁂

1. https://oneuptime.com/blog/post/2026-01-16-docker-multiple-compose-files/view
2. https://stackoverflow.com/questions/70798717/mount-volume-from-host-in-dockerfile-long-format
3. https://lours.me/posts/compose-tip-079-compose-run-advanced/
4. https://docs.docker.com/reference/cli/docker/compose/run/
5. https://docs.docker.com/reference/compose-file/merge/
6. https://docs.docker.com/reference/compose-file/services/
7. https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/
8. https://docs.docker.com/reference/cli/docker/compose/
9. https://docs.docker.com/reference/compose-file/
10. https://docs.docker.com/reference/cli/docker/compose/up/
11. https://devops.stackexchange.com/questions/6246/when-would-i-use-docker-composes-service-ports-flag
12. https://envmanager.com/blog/docker-compose-multiple-files
13. https://docker.ubitools.com/reference/cli/docker/compose/ps/
14. https://stackoverflow.com/questions/74146060/docker-compose-map-multiple-files-or-directories-as-volume
15. https://subscription.packtpub.com/book/cloud-and-networking/9781803234366/9/ch09lvl1sec66/combining-multiple-compose-files-into-one
16. https://frasermclean.com/posts/docker-compose-environment-overrides